



SAPIENZA
UNIVERSITÀ DI ROMA

Progettazione di un sistema hardware/software di diagnostica, acquisizione dati e telemetria real-time di una vettura da corsa della Formula SAE

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Ingegneria Informatica e Automatica

Mark Dynrad Luistro Maranan

Matricola 2066149

Tutore Universitario
Prof. Giorgio Grisetti

Tutore Interno
Prof. Leone Martellucci

Anno Accademico 2025/2026

**Progettazione di un sistema hardware/software di diagnostica, acquisizione dati
e telemetria real-time di una vettura da corsa della Formula SAE**
Sapienza Università di Roma

© 2026 **Mark Dynrad Luistro Maranan**. Tutti i diritti riservati

Questa tesi è stata composta con L^AT_EX e la classe Sapthesis.

Email dell'autore: markdynrad@gmail.com

Sommario

La presente tesi documenta l'attività di progettazione e sviluppo svolta durante il tirocinio presso la scuderia universitaria **Sapienza Fast Charge Formula Student Electric** dell'Università di Roma "La Sapienza".

La Formula Student è una competizione universitaria internazionale organizzata dalla Society of Automotive Engineers (SAE) che prevede la progettazione e la produzione di un'auto da corsa monoposto, valutata sia mediante prove sulla qualità del design che sull'efficienza e sulle prestazioni della vettura. Diffusa in tutto il mondo con decine di eventi mondiali ogni anno, i team devono costruire, testare e promuovere la propria vettura, affrontando sfide ingegneristiche e di gestione del progetto.

Per rispondere ai requisiti di sicurezza e consentire l'analisi dinamica del veicolo, il progetto ha avuto come obiettivo principale la progettazione di un'architettura hardware e software modulare per la diagnostica, l'acquisizione dati e la telemetria real-time. Nello specifico, il contributo si è articolato su quattro ambiti:

- **Firmware per l'Accumulator Management System (AMS):** sviluppo del software bare-metal su microcontrollore automotive dual-core per la supervisione sicura dell'accumulatore ad alta tensione e la stima dello stato di carica (SoC).
- **Sistema di telemetria IoT su ESP32-S3:** progettazione elettronica della scheda (PCB) e del firmware basato su FreeRTOS per acquisire i dati della vettura e trasmetterli ai box o vari dispositivi, anche remoti.
- **Interfaccia Uomo-Macchina (HMI):** ottimizzazione della PCB del volante tramite modulo CFE18 e sviluppo della scheda Dashboard (ATmega328P) con display Nextion per fornire al pilota informazioni essenziali e attivare le spie di sicurezza.
- **Acquisizione dati aerodinamici (Aero Rake):** realizzazione dell'elettronica per campionare 30 sensori di pressione differenziale e trasmettere le grandezze sul CAN bus per la validazione delle simulazioni CFD.

Le soluzioni adottate sono state validate sia a banco che durante i test nelle verifiche tecniche ufficiali di Formula Student Spain e Formula SAE Italy, superando con successo queste ultime in pista, all'Autodromo di Varano de' Melegari,

Indice

1	Introduzione	1
1.1	Contesto Operativo: Formula SAE	1
1.2	Inquadramento del Lavoro Svolto	2
1.3	Struttura del Documento	3
2	Stato dell'Arte e Tecnologie Utilizzate	4
2.1	Vincoli Normativi Formula SAE 2026	4
2.1.1	Specifiche per AMS e Cockpit	4
2.1.2	Specifiche sulla Telemetria Wireless	5
2.2	Protocolli di Comunicazione	5
2.3	Architetture e Funzioni di un BMS Automotive	6
2.3.1	Funzioni di Sicurezza e Gestione dei Guasti	7
2.3.2	Algoritmi per la Stima dello Stato di Carica (State of Charge - SoC)	7
2.4	Sistemi di Telemetria IoT e Acquisizione Dati nel Motorsport	8
2.4.1	Canali di Trasmissione Wireless: Tecnologie e Compromessi	9
2.4.2	Infrastrutture di Visualizzazione Dati e Strumenti di Analisi Real-Time	9
2.5	Ecosistemi Hardware e Software Impiegati	10
3	Architettura del Sistema e Implementazione	11
3.1	Architettura della Monoposto	11
3.1.1	Suddivisone del Traffico CAN	12
3.1.2	Nodi di Calcolo Principali	12
3.2	Firmware dell'Accumulator Management System (AMS)	12
3.2.1	Architettura Dual-Core e Meccanismo di Cross-Watchdog	13
3.2.2	Macchina a Stati Finiti e Attivazione/Disattivazione AIR	13
3.2.3	Controlli di Sicurezza e Rispetto dei Vincoli	14
3.2.4	Algoritmo Ibrido del SoC Estimation e Persistenza su EEPROM	14
3.2.5	Gestione Multi-Bus CAN e Baud Rate Dinamico	16
3.3	Sistema di Telemetria IoT su ESP32-S3	16
3.3.1	Evoluzione rispetto al Sistema Legacy	16
3.3.2	Progettazione Hardware della Scheda Elettronica Ausiliaria	17
3.3.3	Architettura Firmware su FreeRTOS SMP	19
3.3.4	Pipeline di Acquisizione CAN e Decodifica dei Segnali	20

3.3.5	Acquisizione Sensoristica Ausiliaria: Interfacciamento I2C con ADC Esterni	22
3.3.6	Pipeline di Trasmissione: Router e Broker HiveMQ	23
3.3.7	Infrastruttura di Terra e Visualizzazione su Foxglove Studio	24
3.4	Interfaccia Pilota (HMI): Volante e Dashboard	26
3.4.1	Scheda Dashboard su ATmega328P	26
3.4.2	Scheda Volante con Modulo CFE18	28
3.4.3	Interfaccia Grafica su Display Nextion 5"	29
3.5	Sistema Sperimentale di Acquisizione Aero Rake	30
3.5.1	Sensoristica e Hardware di Acquisizione	30
3.5.2	Firmware e Integrazione CAN	31
4	Validazione Sperimentale e Risultati	32
4.1	Validazione a Banco e Testing dell'AMS	33
4.1.1	FSAE Inspections	35
4.2	Prestazioni della Telemetria in Pista	35
4.3	Risultati dell'Interfaccia Cockpit e Volante	37
4.4	Risultati del Sottosistema Aero Rake	38
5	Conclusioni e Sviluppi Futuri	39
5.1	Linee Guida per gli Sviluppi Futuri	39
5.2	Considerazioni Personali	40
	Bibliografia	41

Capitolo 1

Introduzione

1.1 Contesto Operativo: Formula SAE

La **Formula SAE** (nota come **Formula Student**) è una competizione ingegneristica universitaria internazionale in cui team multidisciplinari concepiscono, sviluppano e collaudano in pista monoposto da corsa ad alte prestazioni. La valutazione dei veicoli si articola in eventi statici (analisi economica, business plan e design review del progetto) ed eventi dinamici: acceleration, skidpad, autocross e Endurance, dopo il superamento delle ispezioni.



Figura 1.1. Formula SAE Logo

Nella categoria **Electric Vehicle** (EV), la presenza di un powertrain elettrico e di un pacco batterie con tensioni fino a 600 V DC introduce vincoli tecnici e normativi stringenti:

- **Eterogeneità dei nodi embedded:** la vettura include microcontrollori automotive dedicati alla sicurezza, SoC ad alte prestazioni per la connettività wireless e microcontrollori ausiliari per la sensoristica. Ciascun nodo richiede

un bilanciamento tra risorse computazionali limitate (memoria RAM, Flash e timer), modelli di esecuzione adatti (dal bare-metal ai sistemi operativi real-time) e la progettazione di circuiti stampati (PCB) dedicati e ottimizzati negli spazi.

- **Vincoli Real-Time e sicurezza funzionale:** il regolamento FSAE impone che eventuali anomalie su celle e temperature vengano individuate entro finestre temporali deterministiche (entro 500 ms o 1000 ms, a seconda del segnale), aprendo la catena di emergenza dello *Shutdown Circuit* o **SDC**.
- **Reti distribuite e telemetria:** la cooperazione tra i vari sottosistemi avviene tramite reti CAN (Controller Area Network) bus. È fondamentale evitare colli di bottiglia o saturazioni del bus per garantire una bassa latenza sia nei cicli di controllo di bordo che nello streaming dei dati verso il muretto box.

In questo scenario, la capacità di acquisire, elaborare e trasmettere in tempo reale i parametri della vettura ai box assume un ruolo strategico: permette di diagnosticare istantaneamente guasti, controllare lo stato operativo dell'accumulatore e consentire agli ingegneri di pista e al pilota di prendere decisioni durante le sessioni di test e le prove dinamiche.

1.2 Inquadramento del Lavoro Svolto

L'attività svolta durante il periodo di tirocinio comprende un approccio completo che si estende dall'elettronica a basso livello fino allo sviluppo del software embedded su microcontrollore e fino alla gestione dello streaming telemetrico:

- **Accumulator Management System (AMS):** sviluppo e calibrazione del firmware bare-metal su microcontrollore automotive ad alta affidabilità STMicroelectronics SPC58EC per il monitoraggio di temperatura e tensione delle celle, il calcolo del SoC e il controllo dei relè di potenza e la gestione degli stati operativi del pacco batterie, come richiesto dai vincoli FSAE;
- **Sistema di Telemetria su ESP32-S3:** progettazione elettronica della scheda PCB (*Auxiliary Control Board*) e del firmware FreeRTOS per il parsing del traffico CAN, decodifica dei segnali analogici e la trasmissione via broker MQTTS a un broker cloud;
- **Infrastruttura di terra:** implementazione di un bridge asincrono in Python per convertire i flussi MQTTS in WebSockets visualizzabili su Foxglove Studio;
- **Interfacce Cockpit e Volante (HMI):** riprogettazione della scheda del volante basata su convertitore CFE18, sviluppo della scheda Dashboard (ATmega328P) con display Nextion e ottimizzazione dell'interfaccia grafica (GUI) a beneficio del pilota;
- **Modulo Sperimentale Aero Rake:** integrazione elettronica di un sistema di acquisizione su base ATmega32U4 per il campionamento di sensori di pressione differenziale su bus CAN.

1.3 Struttura del Documento

La tesi è organizzata in cinque capitoli:

- Il **Capitolo 2** presenta lo stato dell'arte, i vincoli normativi FSAE e le tecnologie hardware e software utilizzate;
- Il **Capitolo 3** descrive in dettaglio l'architettura di bordo e l'implementazione hardware e software di tutti i sottosistemi sviluppati;
- Il **Capitolo 4** illustra i risultati delle prove di laboratorio a banco, delle ispezioni di gara e della validazione dinamica su tracciato;
- Il **Capitolo 5** conclude il documento con un bilancio delle attività e le prospettive per i futuri sviluppi tecnici.

Capitolo 2

Stato dell'Arte e Tecnologie Utilizzate

2.1 Vincoli Normativi Formula SAE 2026

All'interno della Formula Student, il regolamento tecnico ufficiale [1] opera come documento formale dei requisiti di sistema, fissando deadline deterministiche e vincoli di tolleranza ai guasti per l'elettronica di bordo [1].

2.1.1 Specifiche per AMS e Cockpit

L'**Accumulator Management System** (AMS) supervisiona costantemente il pacco batterie (regola EV 5.8.1). Il firmware deve rispettare vincoli deterministici:

- **Campionamento continuo (EV 5.8.3):** lettura ininterrotta delle tensioni di tutte le celle, della corrente complessiva e della temperatura di almeno il 30% delle celle.
- **Tempi massimi di intervento (EV 5.8.4, EV 5.8.10):** con una temperatura limite di 60°C, l'AMS deve aprire lo *Shutdown Circuit* (SDC) se un parametro resta fuori scala per oltre **500 ms** (tensione e corrente) o per oltre **1 s** (temperatura).
- **Diagnostica e Latching (EV 5.8.13, EV 6.1.6):** Il firmware deve rilevare la disconnessione dei conduttori di sensing (*open-wire detection*) aprendo l'SDC. Qualsiasi allarme AMS deve rimanere memorizzato "**latched**" e deve poter essere ripristinato solo manualmente dall'esterno dell'abitacolo.
- **Interfaccia di diagnostica (EV 5.8.15):** Il sistema deve esporre una schermata d'ispezione aggregata con tutte le misure per le verifiche tecniche dei commissari.
- **Segnalazioni Cockpit e bulb-check (EV 5.8.12, EV 6.3.8, EV 4.10.10, T 11.9.6):** L'interfaccia pilota richiede spie visibili in luce solare per allarme "AMS" (rossa), guasto isolamento "IMD" (rossa) e stato di sicurezza "TS off" (verde). All'avvio, System Critical Signals (*SCS*) devono eseguire un'accensione di test ("bulb-check" o "blink") forzata compresa tra 1 s e 3 s.

2.1.2 Specifiche sulla Telemetria Wireless

La regola **D 2.2.1** vieta l'invio di segnali di comando o modifiche di parametri verso la vettura una volta varcata la linea di partenza. È consentita unicamente la telemetria unidirezionale (*one-way telemetry*) in downlink per l'analisi dei dati ai box, riservando l'unico link radio bidirezionale al pulsante di emergenza obbligatorio (*RES*).

2.2 Protocolli di Comunicazione

Per coniugare velocità di trasmissione, isolamento e sicurezza, il sistema integra diversi standard di comunicazione:

- **CAN bus (ISO 11898):** principale via di comunicazione, a 500 KBPS. Utilizza un arbitraggio non distruttivo **CSMA/CR** basato su ID, garantendo priorità ai messaggi critici con verifica CRC, prevenendo collisioni sui frame critici di sicurezza e gestione automatica dello stato di *Bus-Off* [2].

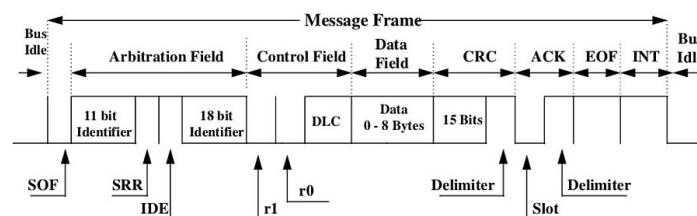


Figura 2.1. Controller Area Network (CAN) 2.0 frame: struttura del messaggio e campi di controllo.

- **isoSPI:** interfaccia differenziale a 2 conduttori basata su trasformatore di isolamento. Converte il segnale SPI garantendo un isolamento galvanico sicuro tra i moduli di monitoraggio ad alto potenziale e l'unità di controllo, consentendo la comunicazione master-slave in daisy-chain[3].
- **Interfacce locali (SPI, I²C, UART):** usate per lo scambio dati tra chip sulla stessa PCB. La linea SPI pilota i controller CAN esterni, la porta I²C connette gli ADC all'ESP32-S3 e la linea UART gestisce il flusso verso il display Nextion.
- **MQTTs (Message Queuing Telemetry Transport su TLS):** protocollo publish/subscribe leggero su stack TCP/IP. Con un overhead di testata minimo (2 byte), disaccoppia la vettura (*Publisher*) dai computer di ricezione ai box (*Subscriber*) attraverso un broker centrale. L'uso della modalità "QoS 0" per lo streaming e della crittografia TLS garantisce aggiornamenti frequenti e sicurezza della connessione[5].

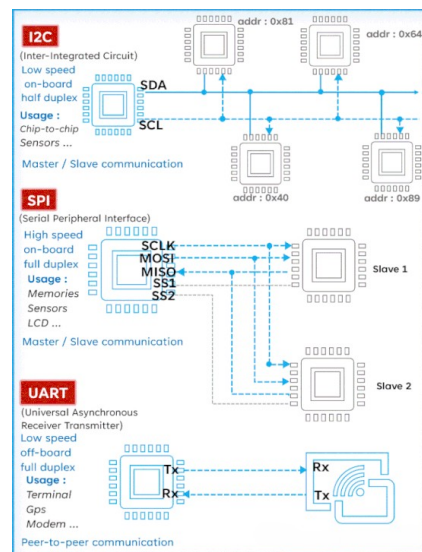


Figura 2.2. I2C, SPI e UART: protocolli di comunicazione seriale

2.3 Architetture e Funzioni di un BMS Automotive

Nei veicoli elettrici ad alte prestazioni, il BMS mantiene ciascuna cella dell'accumulatore all'interno della propria "Safe Operating Area" (SOA), evitando sovratensioni, sovratemperature e degrado precoce [3].

Le applicazioni industriali classificano le architetture hardware dei BMS in due macro-categorie: centralizzata, modulare e distribuita (o Master-Slave).

- **Architettura Centralizzata:** Prevede un'unica scheda elettronica dotata di un microcontrollore e circuiti analogici multicanale che interconnettono direttamente tutte le singole celle dell'accumulatore. Sebbene riduca i costi e i componenti attivi, questa configurazione comporta una notevole complessità del cablaggio analogico, suscettibile a diafonia e rumore elettromagnetico, ed è scarsamente flessibile per pacchi batteria ad alta tensione suddivisi fisicamente in segmenti.
- **Architettura Distribuita / Master-Slave:** Il sistema viene suddiviso in nodi periferici (*Slave* o *Cell Monitoring Boards*) e un'unità di controllo principale (*Master Controller*).

Nelle applicazioni Formula Student si adotta la topologia "**Master-Slave distribuita**": nodi periferici integrano circuiti analogici dedicati ("**Analog Front-End**", **AFE**) montati a ridosso dei moduli fisici per leggere tensioni e temperature, mentre l'unità centrale (**Master**) coordina la macchina a stati di sicurezza, il pilotaggio dei contattori principali di potenza ("**AIR**", **Accumulator Isolation Relays**), il calcolo dello stato di carica (SoC) e garantisce la conformità con la suddivisione meccanica ed elettrica prescritta dal regolamento (EV 5.3.2).

2.3.1 Funzioni di Sicurezza e Gestione dei Guasti

La progettazione del firmware per il Master Controller del BMS adotta paradigmi di sicurezza funzionale. L'architettura software è strutturata su macchine a stati finiti deterministiche (FSM) con compiti di:

- **Supervisione continua delle soglie:** confronta i valori campionati dagli AFE con i limiti critici di *Over-Voltage* (OV), *Under-Voltage* (UV), *Over-Temperature* (OT) e *Over-Current* (OC). Per evitare scatti intempestivi dovuti a transienti o spike di commutazione, l'attivazione degli allarmi è regolata da finestre temporali di persistenza del guasto rigidamente calibrate sulle deadline del regolamento (EV 5.8.10).
- **Diagnostica delle linee di sensing (Open-Wire Detection):** esegue routine di verifica dell'integrità delle connessioni per distinguere una cella a zero volt da un conduttore tranciato, provocando l'apertura cautelativa dell'SDC.
- **Isolamento e Latching dei guasti:** L'interfaccia di attuazione dello Shutdown Circuit è pilotata secondo una logica fail-safe: in caso di blocco del processore, reset imprevisto o timeout del *watchdog timer* interrompe lo Shutdown Circuit.

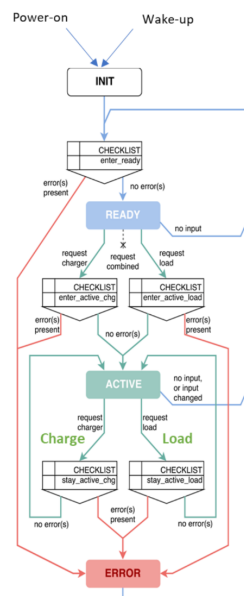


Figura 2.3. FSM del BMS commerciale n-BMS [11]

2.3.2 Algoritmi per la Stima dello Stato di Carica (State of Charge - SoC)

La conoscenza in tempo reale dello stato di carica (SoC) è indispensabile per la gestione delle strategie energetiche in gara (Endurance) e per preservare le celle

da scariche profonde o sovraccariche. I metodi descritti in letteratura variano in complessità computazionale e accuratezza [4]:

- **Open-Circuit Voltage (OCV) Lookup:** Sfrutta la relazione non lineare tra la tensione a circuito aperto della chimica di cella e il SoC residuo. Sebbene sia estremamente efficiente sotto il profilo dell'elaborazione, questo metodo è applicabile solo a veicolo fermo o dopo un adeguato tempo di rilassamento della cella, risultando inefficace durante l'erogazione continua o la frenata rigenerativa a causa delle cadute ohmiche e delle tensioni di polarizzazione.
- **Coulomb Counting (Integrazione della Corrente):** Calcola la carica istantanea integrando nel tempo la corrente misurata sul bus ad alta tensione:

$$SoC(t) = SoC(t_0) - \frac{1}{C_n} \int_{t_0}^t I(\tau) d\tau \quad (2.1)$$

dove C_n rappresenta la capacità nominale della cella e $I(\tau)$ la corrente misurata. Il principale limite risiede nell'accumulazione dell'errore (drift) causato dall'offset e dal rumore di quantizzazione dell'ADC del sensore di corrente.

- **Filtri di Kalman Estesi (EKF) e Osservatori Adattivi:** Consentono di fondere i modelli elettrici equivalenti della cella con le misurazioni di tensione e corrente, correggendo ricorsivamente gli errori di stima. Tuttavia, richiedono una notevole potenza di calcolo (algebra matriciale a virgola mobile) e un'accurata parametrizzazione termica ed elettrochimica del modello di cella.

Nelle applicazioni Formula Student su microcontrollori dedicati, l'approccio computazionalmente più bilanciato combina l'integrazione di Coulomb in marcia con la calibrazione periodica del punto iniziale $SoC(t_0)$ basata su curve OCV a vettura ferma, memorizzando lo stato calcolato in memoria non volatile per garantirne la persistenza tra i cicli di accensione della bassa tensione.

2.4 Sistemi di Telemetria IoT e Acquisizione Dati nel Motorsport

Nelle moderne competizioni automobilistiche, l'acquisizione dei dati si è evoluta dal logging offline su supporti fisici (SD card) a sistemi integrati di *Edge Computing* e telemetria wireless real-time in grado di trasmettere i flussi telemetrici della monoposto verso la corsia box [2, 5].

In una vettura da corsa elettrica, l'architettura di acquisizione telemetrica opera secondo una pipeline a tre livelli gerarchici:

1. **In-Vehicle Network (Fieldbus Layer):** Le unità elettroniche di bordo (BMS, Inverter, VCU, Dashboard, sensori aerodinamici) immettono sul CAN bus centinaia di parametri dinamici ed elettrici a frequenze comprese tra 10 Hz e 1 kHz.
2. **Edge Telemetry Gateway (Processing Layer):** Un nodo dedicato (nella monoposto implementato tramite SoC ESP32-S3) agisce da ponte tra la rete cablata di bordo e il canale radio.

3. **Ground Station & Ingestion Layer (Pit/Cloud Layer):** I pacchetti ricevuti dal canale wireless vengono acquisiti da un broker centrale situato al muretto box, il quale distribuisce i dati e applicativi di visualizzazione real-time.

2.4.1 Canali di Trasmissione Wireless: Tecnologie e Compromessi

La definizione del canale radio per la telemetria deve bilanciare requisiti contrapposti di ampiezza di banda, latenza, affidabilità del collegamento e complessità dell'infrastruttura di terra. Nella Formula Student si usano due approcci principali:

- **Infrastruttura Wi-Fi Dedicata Vettura-Box (IEEE 802.11 b/g/n):** Prevede un collegamento diretto peer-to-peer tra il ricetrasmittitore di bordo e un access point posizionato al muretto box. Sebbene garantisca throughput elevati in prossimità della corsia box, questa soluzione presenta criticità intrinseche sui tracciati estesi: la portata utile in radiofrequenza è fortemente attenuata da ostacoli visivi, dislivelli e barriere metalliche di contenimento, generando frequenti dead zones e intermittenti perdite di segnale.
- **Reti Cellulari ad Ampia Copertura (4G-LTE):** Sfruttano le stazioni radio base commerciali per instradare i flussi dati verso un'infrastruttura Internet remota. L'approccio cellulare garantisce continuità di copertura lungo l'intero sviluppo del tracciato, azzerando la necessità di dispiegare apparati d'antenna nei paddock.

2.4.2 Infrastrutture di Visualizzazione Dati e Strumenti di Analisi Real-Time

L'utilità della telemetria in pista si mette in parallelo all'analisi post-sessione, in modo tale da avere un overview della vettura in moto. L'analisi post-sessione si affida tradizionalmente a software dedicati all'elaborazione dei file di log (es. MATLAB), il monitoraggio dinamico al muretto richiede suite di visualizzazione modulari capaci di gestire streaming asincroni ad alta frequenza (es. Foxglove Studio o dashboard basate su WebSockets). Tali piattaforme permettono agli ingegneri di definire pannelli personalizzati.

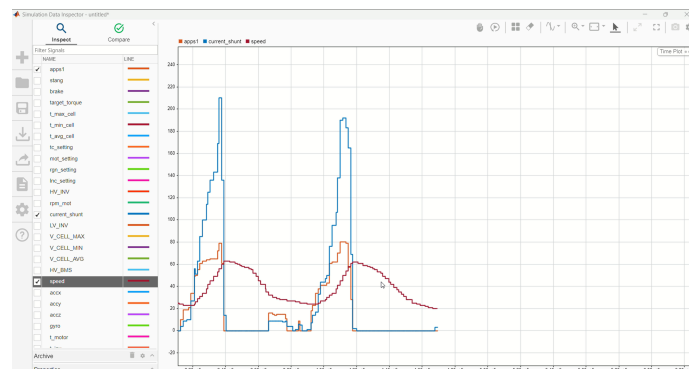


Figura 2.4. Analisi Real Time dei dati attraverso Matlab Data Inspector

2.5 Ecosistemi Hardware e Software Impiegati

I nodi di bordo usano le seguenti piattaforme:

- **Controllo Batteria (SPC58EC80E5):** Microcontrollore automotive a 32 bit dual-core (frequenza fino a 180 MHz, conforme ISO 26262 ASIL-D) sviluppato in C nativo con toolchain GCC tramite l'ambiente "SPC5Studio" [6]. Flusso deterministico strutturato su superloop e temporizzazione guidata da interrupt timer periodici. Interfacciato ad AFE STMicroelectronics L9963E [7].
- **Gateway di Telemetria (ESP32-S3-N16R8):** SoC dual-core a 240 MHz con 512 KB di SRAM, 16 MB di Flash e 8 MB di Octal PSRAM. Firmware sviluppato su framework "ESP-IDF" e scheduler "FreeRTOS SMP" [8]. Architettura multi-tasking simmetrica. Il "Core 1" gestisce il campionamento su bus CAN/TWAI a 500 kbps, il filtraggio dati e periferiche secondarie; il "Core 0" gestisce lo stack di rete LwIP e la trasmissione wireless.



Figura 2.5. ESP32-S3-N16R8 SoC by Espressif Systems [8]

- **Display e Comandi Cockpit:** Scheda Dashboard custom basata su microcontrollore a 8 bit "ATmega328P" e controller CAN "MCP2515" [9], interfacciata a display TFT touch Nextion Intelligent da 5". Scheda volante custom integrata con modulo "CFE18 CAN Switchboard" che digitalizza gli ingressi del pilota (motor map, traction control, regen) trasmettendoli su bus CAN.
- **Sensori Aerodinamici (Aero Rake):** Nodo di misura compatto basato su "ATmega32U4" (Arduino Micro) con modulo di campionamento analogico a 10 bit e controller CAN MCP2515, per l'acquisizione di sensori di pressione analogici.
- **Infrastruttura Box:** broker cloud HiveMQ, bridge asincrono in Python e suite Foxglove Studio per l'analisi telemetrica real-time.

Capitolo 3

Architettura del Sistema e Implementazione

3.1 Architettura della Monoposto

La monoposto rappresenta un sistema distribuito con vincoli temporali eterogenei: dai compiti critici dell'inverter e del pacco batterie fino ai flussi asincroni di telemetria e guida autonoma. Per prevenire fenomeni di interferenza tra i flussi di dati e isolare i messaggi critici legati alla trazione e alla sicurezza dell'accumulatore, l'architettura di bordo è strutturata su una rigida suddivisione delle reti, schematizzata nella Figura 3.1.

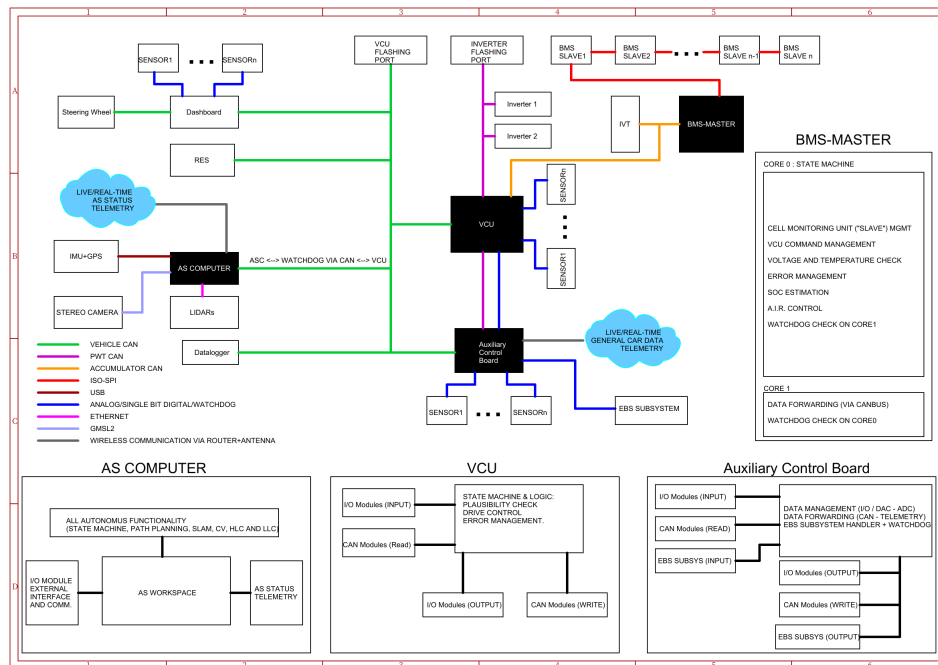


Figura 3.1. Schema a blocchi dell'architettura hardware, dei nodi di elaborazione e delle reti di comunicazione della monoposto.

3.1.1 Suddivisione del Traffico CAN

La dorsale di rete è strutturata su tre linee CAN a 500 kbps e una dorsale dedicata interna al pacco batterie:

- **Vehicle CAN (500 kbps):** Costituisce il canale di comunicazione principale che connette la Vehicle Control Unit (VCU), la Dashboard, il volante, la telemetria, il Datalogger ufficiale e l'AS Computer;
- **Powertrain CAN (500 kbps):** rete punto-a-punto isolata per il controllo di coppia e rigenerazione tra la VCU e gli inverter di trazione;
- **Accumulator CAN (Dual-Rate):** linea tra BMS Master, sensore di corrente IVT e VCU. Opera a **500 kbps** durante la marcia e commuta automaticamente a **125 kbps** quando il veicolo è collegato al caricabatterie esterno. L'AMS riconfigura il modulo FlexCAN alla velocità fissa di 125 kbps, parametro imposto dal protocollo proprietario del caricabatterie industriale impiegato dal team.

3.1.2 Nodi di Calcolo Principali

Le funzioni logiche sono ripartite tra centraline mirate:

- **VCU (Ecotron):** gestisce gli stati vettura, la plausibilità della pedaliera e l'algoritmo di ripartizione della coppia (*Torque Vectoring*);
- **BMS Master (SPC58EC):** controlla gli AIR, esegue la diagnostica delle celle e misura il SoC in tempo reale;
- **Auxiliary Control Board (ESP32-S3):** acquisisce la sensoristica ausiliaria di telaio, gestisce i watchdog dell'impianto frenante di emergenza (*EBS*) e inoltra i dato verso il router 4G di bordo;
- **Cockpit e Volante:** il modulo CFE18 sul volante acquisisce i comandi del pilota, mentre la Dashboard (ATmega328P) pilota le spie di sicurezza e inoltra i dati al display Nextion;
- **Aero Rake (ATmega32U4):** nodo ausiliario rimovibile, connesso al Vehicle CAN per registrare la scia aerodinamica durante sessioni di test dedicate [10].

3.2 Firmware dell'Accumulator Management System (AMS)

L'*Accumulator Management System* (AMS) rappresenta l'unità logica preposta alla sicurezza e alla supervisione dell'accumulatore ad alta tensione. Il firmware è stato sviluppato in C nativo per il microcontrollore automotive a 32 bit "STMicroelectronics SPC58EC", operante a una frequenza di sistema di 180 MHz.

3.2.1 Architettura Dual-Core e Meccanismo di Cross-Watchdog

E' stata adottata un'architettura rigorosamente **bare-metal** priva di sistemi operativi real-time basata sulla suddivisione dei carichi computazionali sui due core fisici :

- **Core Primario (Core 2, task 50 ms):** esegue la macchina a stati (FSM), legge i 16 slave AFE via isoSPI, verifica le soglie di tensione e temperatura, calcola il SoC e pilota i relè AIR;
- **Core Secondario (Core 0, task 20 ms):** formatta e trasmette i frame periodici e diagnostici sulle interfacce CAN.

Per verificare che nessun core rimanga bloccato, è stato implementato un meccanismo di controllo reciproco (*Cross-Watchdog*) basato sulla periferica hardware SEMA4 (Codice 3.1). Attraverso le funzioni `sema4MainCore()` e `sema4SecCore()`, ciascun core acquisisce atomicamente il proprio semaforo hardware (`sem_c1` e `sem_c2`) per registrare il timestamp del proprio ciclo (`cycletime`). Contestualmente, ciascun core interroga il timestamp aggiornato dall'altro core: se il ritardo temporale supera il timeout di sicurezza fissato a `TIMEOUT_CORE_WATCHDOG = 5000 ms`, viene forzata l'invocazione di `SPCSetRunMode(15)`, transizionando il microcontrollore nel reset mode di sicurezza hardware.

Listing 3.1. Funzione di verifica del Cross-Watchdog su semaforo hardware SEMA4.

```

1 static uint8_t sema4MainCore(void) {
2     uint32_t timestamp_wait;
3     res = sema4_lock(sem_c1);
4     while (res != SEMA4_NO_ERROR){};
5     cycletime_c1 = getMillis();
6     res = sema4_unlock(sem_c1);
7     while (res != SEMA4_NO_ERROR){};
8
9     timestamp_wait = getMillis();
10    do {
11        if (sema4_islocked(sem_c2) == 0) {
12            if (delta_millis(getMillis(), cycletime_c2) > TIMEOUT_CORE_WATCHDOG)
13                return 0; // Timeout rilevato sull'altro core
14            else return 1;
15        }
16    } while (delta_millis(getMillis(), timestamp_wait) < TIMEOUT_CORE_CHECK);
17    return 0;
18 }
```

3.2.2 Macchina a Stati Finiti e Attivazione/Disattivazione AIR

Il funzionamento dell'AMS segue quattro stati operativi:

- **START:** porta a livello logico alto l'abilitazione dello Shutdown Circuit e numerica dinamicamente i nodi slave (`AFEaddressing()`). Se vengono riconosciuti tutti i 16 moduli previsti, passa a **READY**;

- **READY:** mantiene aperti i relè di potenza (`AIR_POS`, `AIR_NEG`, `AIR_PREC`). Esegue la verifica dei parametri (`Check()`) e calcola il punto di lavoro del SoC (`getSOC()`). Su comando del pilota (`AIR_CMD`), dopo un debouncing di 180 ms, passa in `ACTIVE`;
- **ACTIVE:** azzerava i fault latenti e coordina la procedura di pre-carica, chiudendo prima `AIR_NEG` e la resistenza di pre-carica (`AIR_PREC`). Trascorsi 3000 ms, a condensatori carichi, chiude `AIR_POS` e disinserisce `AIR_PREC`. In caso di anomalie, commuta immediatamente in `ERROR`. Durante la marcia, se il comando pilota cambia (`AIR_OPENED`), la FSM retrocede in `READY`; in presenza di qualsiasi anomalia rilevata da `Active_Check()`, commuta istantaneamente in `ERROR`;
- **ERROR:** azzerava il pin di comando aprendo la catena di sicurezza esterna e diseccitando tutti i relè. La chiusura richiede che tutti i parametri rientrino nella norma per almeno 2000 ms consecutivi per rientrare in `READY`, altrimenti in `START` se l'inizializzazione non è stata mai completata o fallita.

3.2.3 Controlli di Sicurezza e Rispetto dei Vincoli

La funzione periodica `Check()` valida contemporaneamente tre condizioni:

$$\begin{aligned} & (V_Checklist() == CELLS_OK) \\ & \wedge (T_Checklist() == TERM_OK) \\ & \wedge (CheckTimeoutCurrIVT() == IVT_OK) \end{aligned} \quad (3.1)$$

1. **V_Checklist():** verifica che ogni cella resti tra 2.450 V e 4.250 V. Eventuali superamenti devono persistere per oltre 500 ms prima di innescare l'arresto, filtrando i disturbi transitori pur garantendo l'intervento entro i limiti di regolamento (EV 5.8.10).
2. **T_Checklist():** verifica che la temperatura delle celle non superi la soglia di 60 °C con una persistenza di 800 ms (sotto il secondo imposto dal regolamento).
3. **TCheckTimeoutCurrIVT() :** se il frame CAN del sensore di corrente (0x521) non viene ricevuto entro 500 ms, viene generato un allarme di timeout azzerando le variabili di corrente.

3.2.4 Algoritmo Ibrido del SoC Estimation e Persistenza su EEPROM

La valutazione dello stato di carica dell'accumulatore combina l'interpolazione della tensione a circuito aperto (*OCV Lookup Table*) per la fase stazionaria con l'integrazione continua della corrente (*Coulomb Counting*) durante l'erogazione di trazione.

L'algoritmo combina due metodi:

1. **Inizializzazione OCV:** all'avvio in `READY`, il sistema stima il SoC confrontando le tensioni a riposo su una tabella di calibrazione a 21 punti. Se lo

scostamento rispetto al dato memorizzato su Flash EEPROM supera il 10%, viene assunto il valore OCV per correggere derive storiche; in caso contrario, viene ereditato il dato persistente per garantire continuità dopo brevi power-cycle.

2. **Coulomb Counting:** durante lo stato **ACTIVE**, ogni lettura di corrente aggiorna il bilancio di carica tramite integrazione discreta:

$$\Delta Q = I \cdot \Delta t \quad (3.2)$$

$$\Delta SOC = \frac{I \cdot \Delta t}{C_{\text{nominale}}} \quad (3.3)$$

Dove I è la corrente istantanea misurata dal sensore IVT, in **mA**, Δt è l'intervallo di campionamento in **ms**, C_{nominale} è la capacità nominale del pacco batteria in **mAh** e ΔSOC (STEP) è adimensionale su scala fissa 0 - 10000 dove (10000 = 100.00%).

$$\Delta Q[mAh] = \frac{I[mA] \cdot \Delta t[ms]}{3600000} \quad (3.4)$$

$$\Delta SOC[Step] = \frac{\left(\frac{I[mA] \cdot \Delta t[ms]}{3600000}\right)}{C_{\text{nominale}}[mAh]} \cdot 10000 \quad (3.5)$$

$$\Delta SOC[Step] = \frac{I[mA] \cdot \Delta t[ms]}{C_{\text{nominale}}[mAh] \cdot 360} \quad (3.6)$$

```

1 // Calcolo Coulomb Counting nel ciclo di ricezione CAN IVT (main.c)
2     if (b.status == ACTIVE){
3         uint32_t delta_t_soc = delta_millis(t.t_ivt, t.t_ivt_last);
4         int64_t raw_delta = ((int64_t)b.current_A * (int64_t)delta_t_soc) +
5             b.delta_SOC_remainder;
6         int64_t den = (int64_t)CAPACITY_MAH * 360LL;
7         if (den > 0) {
8             b.delta_SOC += (int32_t)(raw_delta / den);
9             b.delta_SOC_remainder = raw_delta % den;
10        }
11    }
    t.t_ivt_last = t.t_ivt;

```

Figura 3.2. Snippet dell'integrazione discreta di Coulomb Counting sincronizzata sul frame CAN del sensore di corrente.

3. **Salvataggio su Memoria Flash:** Ogni 60 s

(SOC_EEPROM_SAVE_FREQ_MS = 60000), il firmware salva il buffer aggiornato in Flash attraverso `EEPROMWrite()`, azzerando il rischio di perdita dello stato di carica in caso di spegnimento improvviso della vettura.

3.2.5 Gestione Multi-Bus CAN e Baud Rate Dinamico

L'AMS gestisce la trasmissione dei parametri a 50 Hz su due controller CAN (CAND2 e CAND4). Vengono trasmessi frame di stato (FSM, comando AIR), allarmi puntuali, grandezze elettriche (tensione pacco, corrente, SoC) e una scansione multiplexata di tutte le 16 schede slave.

Tabella 3.1. Matrice dei frame CAN periodici trasmessi dall'AMS (Core 0).

CAN ID	DLC	Segnali e Grandezze Trasmesse
0x010	8	Heartbeat
0x0FF	8	Bitmask allarmi
0x0FE	8	Diagnostica Errori
0x620	8	Stato operativo
0x621	7	Valori termici
0x622	8	Valori elettrici Compelssivi
0x623	6	Valori voltmetrici
0x625	4	Indici di cella relativi a valori voltmetrici
0x6A1–0x746	8	Mappatura per diagnostica di tutte le 16 CMU (EV 5.8.15)

All'avvio, il firmware controlla lo stato dei pin di modalità: se è presente il segnale di ricarica (CHARGER_SENSING), il controller commuta deterministicamente il baud rate da "500 kbps" a "125 kbps" per dialogare con l'interfaccia proprietaria del caricabatterie esterno.

3.3 Sistema di Telemetria IoT su ESP32-S3

La *Auxiliary Control Board* (ACB) integra le funzioni di gateway di telemetria, acquisizione e gestione di sensori ausiliari e interfaccia di sicurezza per il sistema frenante di emergenza (EBS).

3.3.1 Evoluzione rispetto al Sistema Legacy

Nelle stagioni precedenti, la trasmissione telemetrica era affidata a un'architettura integrata all'interno della vecchia scheda Dashboard, basata su un microcontrollore ESP32 standard (Classic). Tale configurazione presentava gravi colli di bottiglia architetturali e operativi che ne limitavano drasticamente l'affidabilità.

- **Accoppiamento Hardware e Single Point of Failure:** La scheda Dashboard richiedeva al medesimo chip sia l'aggiornamento della grafica del display Nextion via UART, sia la ricezione CAN, sia la gestione dello stack radio. Un blocco software del microcontrollore causava contestualmente l'azzeramento della visualizzazione per il pilota e la perdita della telemetria ai box.
- **Framework Monolitico su Arduino Core (.ino):** Il firmware legacy era sviluppato all'interno dell'ambiente Arduino. L'overhead tipico delle librerie wrapper per Arduino introducevano jitter imprevedibili, impedendo un controllo totale sulle priorità dei core di calcolo e sulla gestione della memoria dinamica.

- **Infrastruttura di Rete Amatoriale (Hotspot e Cloudflared):** Il transito dei pacchetti verso l'esterno sfruttava un hotspot Wi-Fi generato da uno smartphone posizionato nell'abitacolo. L'ESP32 apriva una connessione WebSocket grezza esposta verso Internet mediante il tunneling inverso *cloudflared*. A terra, uno script Python intercettava il tunnel HTTP/WebSocket e inoltrava i dati verso un modello MATLAB/Simulink per la visualizzazione grafica. Questa catena risultava fragile: le frequenti disconnessioni dello smartphone provocavano la rottura del tunnel *cloudflared*, l'impossibilità di supportare accessi concorrenti da più postazioni box e l'assenza di un broker dedicato con code di persistenza.
- **Saturazione della Memoria RAM:** L'ESP32 Classic disponeva unicamente di circa 320 KB di SRAM interna utilizzabile. L'allocazione contemporanea dei buffer di ricezione CAN, dei frame buffer di trasmissione e dei socket di rete portava frequentemente a condizioni di *heap exhaustion*, con conseguente riavvio spontaneo del microcontrollore tramite panic di sistema.

Tabella 3.2. Confronto sinottico tra l'architettura telemetrica legacy e la soluzione ingegnerizzata su Auxiliary Control Board.

Parametro / Layer	Architettura Legacy	Nuova Architettura
MCU	ESP32 Classic (Dual Core 240 MHz)	ESP32-S3-N16R8 (Dual Core Xtensa LX7)
Framework Software	Arduino IDE (.ino), super-loop bloccante	ESP-IDF nativo su FreeRTOS dual-core SMP
Memoria Volatile	~320 KB SRAM interna	512 KB SRAM + 8 MB Octal PSRAM esterna
Suddivisione Compiti	Telemetria + Rendering HMI congiunti	Nodo ACB dedicato (HMI delegata su ATmega)
Connettività RF Bordo	Hotspot Wi-Fi	Router automotive 4G-LTE dedicato
Protocollo Applicativo	WebSocket Server su MCU + tunnel <i>cloudflared</i>	MQTTs su TLS (porta 8883) via HiveMQ
Pipeline di Terra	Script Python → Simulink	Python Bridge → Foxglove Studio
Throughput / Scalabilità	Limitato a 1 client, frequenti freeze	Multi-subscriber scalabile

Come riassunto nella Tabella 3.2, la nuova scheda separa completamente i compiti di telemetria da quelli della scheda Dashboard.

3.3.2 Progettazione Hardware della Scheda Elettronica Ausiliaria

L'elettronica di bordo della scheda ACB è progettata per garantire elevata immunità ai transienti condotti e operare con affidabilità all'interno della rete a bassa tensione del veicolo (LVS a 12 V nominali):

- La scheda converte la linea a 12 V proveniente da una scheda di alimentazione nei domini di alimentazione logica a 5 V e 3.3 V. La conversione primaria è

affidata a un Buck Converter 12 V a 5 V, protetto a monte da un circuito contro l'inversione di polarità (mediante diodo Schottky e e-Fuse) e da soppressori di transienti di tensione (*TVS*). L'alimentazione a 3.3 V per il SoC ESP32-S3 è ulteriormente stabilizzata da uno stadio LDO a basso rumore, con estensivo disaccoppiamento capacitivo (condensatori ceramici MLCC in package compatto posizionati a ridosso dei pin V_{DD}).

- Il controllore CAN periferico integrato nell'ESP32-S3 (interfaccia TWAI/SJA1000) è accoppiato a un transceiver CAN TJA1051, compatibile ISO 11898-2, per la comunicazione con la rete CAN VEHICLE. Viene unito a una seconda interfaccia CAN INVERTER, attraverso un MCP2515 con transceiver CAN TJA1051.

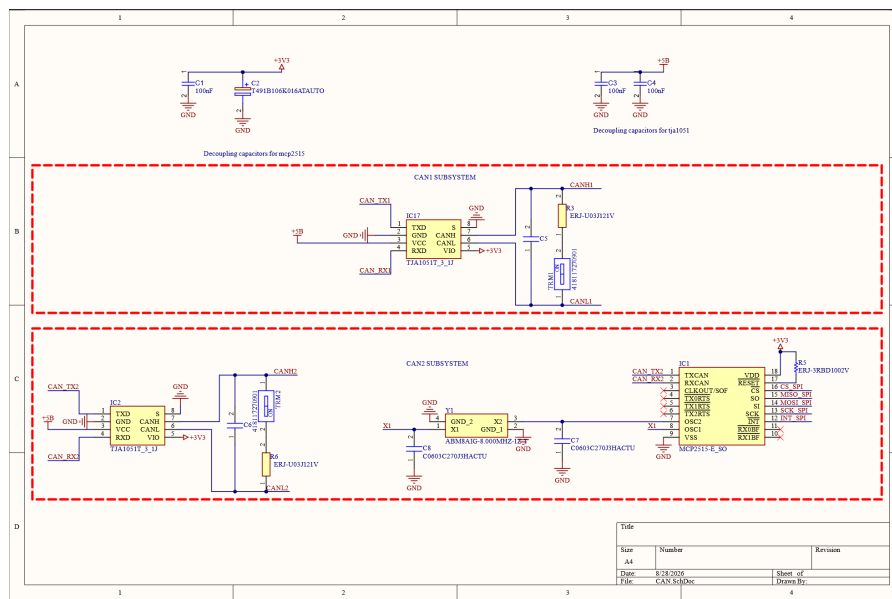


Figura 3.3. Dettaglio del layout della PCB ACB con evidenziati i sottosistemi CAN

- La scheda alloggia i circuiti secondari che includono traslatori di livello logico e buffer per comunicazione e controllo del sottosistema frenante autonomo (*EBS Subsystem*), ingressi protetti con filtro RC passa-basso per la lettura di canali digitali e analogici ausiliari. La scheda alloggia la componentistica necessaria alle funzioni secondarie di controllo: stadi di commutazione a MOSFET di potenza per l'attuazione dell'EBS di sicurezza driverless, ingressi analogici filtrati per canali ausiliari e circuiti di condizionamento per encoder di posizione su ingressi dedicati.
- un piano di massa continuo a 4 strati con opportuna *keep-out zone* attorno all'antenna a circuito stampato dell'ESP32-S3.

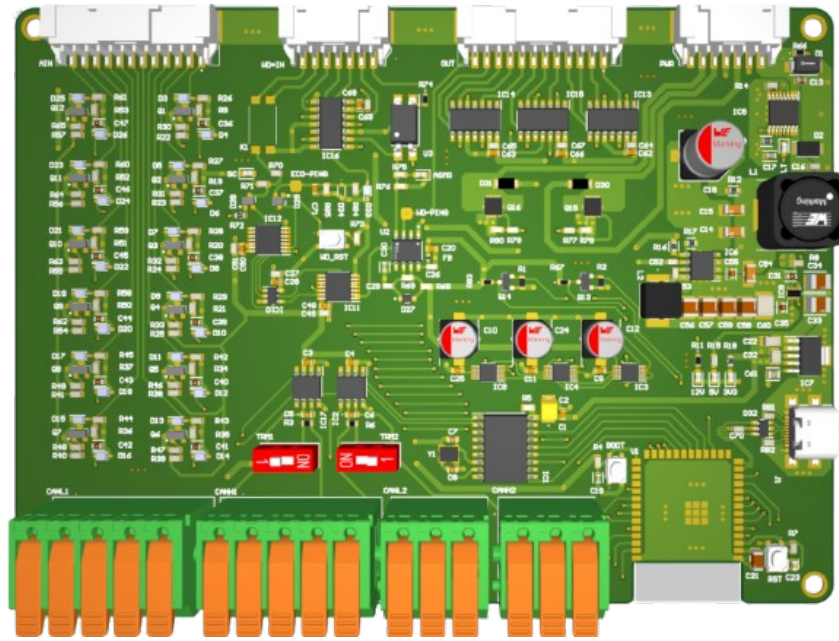


Figura 3.4. Render 3D e layout della PCB Auxiliary Control Board basata sul SoC ESP32-S3-N16R8.

3.3.3 Architettura Firmware su FreeRTOS SMP

Il firmware è sviluppato nativamente sull'*Espressif IoT Development Framework* (ESP-IDF v5.5.x) in linguaggio C. L'architettura software sfrutta il supporto Symmetric Multiprocessing (SMP) per vincolare i task operativi ai due core fisici Xtensa LX7 (Core 0 e Core 1):

- **Core 1 (Gestione Rete CAN):** Allocated all'esecuzione del task prioritario di ricezione CAN/TWAI/MCP2515 `can_manager_task`. Questo core gestisce il prelievo immediato dei frame dal buffer hardware, la validazione dei messaggi, immettendo i pacchetti validati all'interno di una coda thread-safe `QueueHandle_t` senza subire ritardi o prelezioni dallo stack di comunicazione.
- **Core 1 (Integrazione delle Periferiche Secondarie):** A supporto dell'operatività del sistema, il firmware include moduli ausiliari gestiti tramite API native ESP-IDF: il driver LEDC (PWM) per il controllo di motori ausiliari (es. Ventole e pompe), il driver PCNT per la cattura di impulsi per sensori rotativi (encoder), il driver I2C per la gestione dei ADC, il driver RMT per la generazione di segnali digitali ad alta risoluzione per il controllo di attuatori stepper e di strisce led esterne indirizzabili.
- **Core 0 (Networking, Formattazione & Trasmissione Telemetria):** Interamente dedicato alla gestione dello stack TCP/IP (LwIP), al mantenimento del link radio Wi-Fi locale verso il router di bordo e all'esecuzione del task telemetrico (`telemetry_mqtt_task`). Il task estrae i dati dalla coda inter-core, struttura i messaggi e invia i payload cifrati verso il broker MQTTs.

Listing 3.2. Esempio di inizializzazione dei task sui due core dell'ESP32-S3.

```
1 static car_data_handler_t c;  
2 static car_data_t car_data = {0};  
3  
4 void app_main(void) {  
5     c.mutex = xSemaphoreCreateMutex();  
6     c.data = &car_data;  
7     init_core1();  
8     init_core0();  
9 }  
10  
11 void init_core1(void) {  
12     init_peripherals();  
13     xTaskCreatePinnedToCore(Peripherals_task, "PERIPH", 4096, NULL,  
14                             configMAX_PRIORITIES - 1, NULL, 1);  
15 }  
16  
17 void init_core0(void) {  
18     init_network();  
19     xTaskCreatePinnedToCore(Network_task, "NET", 8192, NULL, configMAX_PRIORITIES -  
20                             2, NULL, 0);  
21 }
```

3.3.4 Pipeline di Acquisizione CAN e Decodifica dei Segnali

La gestione delle linee CAN è affidata a due controller distinti: il controller TWAI integrato nell'ESP32-S3 per la rete Vehicle CAN e un controller esterno MCP2515 per la rete Inverter. Entrambi i controller operano a 500 kbps, con buffer hardware di ricezione e trasmissione configurati per garantire la continuità dei dati.

Unico modulo per la Gestione delle reti CAN: Viene sviluppato un modulo unificato per la gestione dei due controller CAN, che implementa una coda thread-safe per la ricezione e la trasmissione dei frame. I task di ricezione e trasmissione sono separati per ciascun controller, garantendo l'isolamento dei flussi di dati e la gestione asincrona degli eventi. I due sottomoduli si interfacciano con il resto del firmware tramite una struttura dati condivisa, protetta da mutex per evitare condizioni di race. Questi sono:

- **Driver TWAI/SJA1000(Vehicle CAN):** Scelto per questa rete per la possibilità di gestione della grandezza dei buffer in ricezione e in trasmissione tramite API native ESP-IDF, flessibilità nella configurazione e nella gestione degli eventi, ulteriormente spinta dallo stato quasi saturo di questa rete. Il driver è configurato per operare in modalità *non-blocking*, con callback di notifica su ricezione frame e gestione automatica degli errori di bus, a 500 kbps.
- **Driver MCP2515 (Inverter CAN):** La libreria open-source per il controller MCP2515 è stata adattata alle API native ESP-IDF, consentendo la gestione dei frame a 500 kbps e la decodifica in strutture dati C. Buffer ridotto ma sufficientemente veloce per decodificare esclusivamente frame per feedback e debug durante la calibrazione dei motori.

Resilienza e Fault Handling del Bus: Il controller monitora costantemente i registri di stato dei transceiver. Qualora il contatore di errori di trasmissione (*TEC*) o ricezione (*REC*) ecceda i limiti inducendo lo stato di *Bus-Off* o *Bus-Heavy*, `can_diag_task()` attiva l'auto-ripristino: quando in stato di errore, invocando le funzioni `twai_can_bus_recovery()` e `mcp2515_reset()`, il driver attende la sequenza standard di 128 occorrenze di 11 bit recessivi consecutivi sul bus e riavvia automaticamente la periferica senza richiedere un reboot hardware della scheda.

Listing 3.3. Snippet delle tasks singole

```
1
2 static void twai_rx_task(void *arg) {
3     for (;;) {
4
5         if (frame_received) {
6             xSemaphoreTake(c.mutex)
7             decode_update_car_data(frame);
8             xSemaphoreGive(c.mutex)
9         }
10    }
11 }
12
13 static void mcp2515_rx_task(void *arg) {
14     for (;;) {
15
16         if (frame_received) {
17             xSemaphoreTake(c.mutex);
18             decode_update_car_data(frame);
19             xSemaphoreGive(c.mutex);
20         }
21    }
22 }
23
24 static void twai_tx_task(void *arg) {
25     for (;;) {
26
27         if (xQueueReceive(twai_tx_queue, &frame)){
28             send_frame_twai(frame);
29         }
30    }
31 }
32
33 static void mcp2515_tx_task(void *arg) {
34     for (;;) {
35
36         if (xQueueReceive(mcp2515_tx_queue, &frame)){
37             send_frame_mcp2515(frame);
38         }
39    }
40 }
```

Listing 3.4. Snippet delle gestione CAN unificata

```

1 void can_init(void) {
2     init_twai_queue();
3     init_mcp2515_queue();
4     xTaskCreate(twai_tx_task, "TWAI_TX");
5     xTaskCreate(twai_rx_task, "TWAI_RX");
6     xTaskCreate(mcp2515_tx_task, "MCP_TX");
7     xTaskCreate(mcp2515_rx_task, "MCP_RX");
8 }
9
10
11 void can_send(CAN_BUS bus, CAN_FRAME frame){
12     if (bus == CAN_VEHICLE) {
13         xQueueSend(twai_tx_queue, &frame);
14     } else if (bus == CAN_INVERTER) {
15         xQueueSend(mcp2515_tx_queue, &frame);
16     }
17 }
18
19 // In ricezione non hanno bisogno di chiamate a funzione, poiche' i task girano
    attraverso scheduling FreeRTOS e gestiscono le code di ricezione in maniera
    asincrona.

```

3.3.5 Acquisizione Sensoristica Ausiliaria: Interfacciamento I²C con ADC Esterni

Il campionamento dei canali analogici ad alta precisione è stato demandato a convertitori analogico-digitali esterni ADS1115 dell'azienda Texas Instruments, interfacciati all'ESP32-S3 tramite il bus seriale a due fili "I²C" (*Inter-Integrated Circuit*):

- **Configurazione Hardware del Bus I²C:** Le linee fisiche seriali (SDA per i dati e SCL per il clock) sono connesse a GPIO dedicati del SoC provvisti di resistenze di pull-up esterne calibrate (4.7 kΩ).

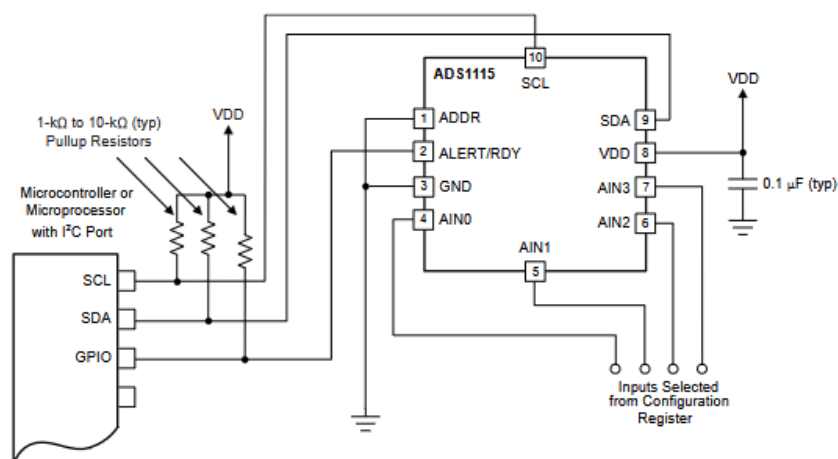


Figura 3.5. Esempio di connessione di un ADC ADS1115 al bus I²C dell'ESP32-S3.

- **Driver ESP-IDF e Gestione delle Transazioni:** A livello software, l'interfacciamento è gestito tramite le API native del driver periferico `driver/i2c.h` di ESP-IDF. Il ciclo di lettura prevede transazioni seriali strutturate:
 1. *Scrittura del Configuration Register:* Selezione del canale di multiplexing analogico, impostazione del fattore di amplificazione dello stadio PGA (*Programmable Gain Amplifier*) e avvio della conversione in modalità *single-shot* o *continuous*;
 2. *Polling/Interrupt di Conversione Completa:* Attesa del completamento del campionamento nel rispetto del tempo di assestamento dell'ADC;
 3. *Lettura del Conversion Register:* Acquisizione atomica dei 2 byte di dato (MSB e LSB) mediante comandi concatenati di start ripetuto (*repeated start*) e acknowledge hardware (ACK).
- **Integrazione nella Pipeline FreeRTOS e Telemetria:** La routine di interrogazione I²C è integrata all'interno del task di elaborazione dati su "Core 1", sincronizzata con timer periodici non bloccanti. L'intero pacchetto di dati viene mandato sulla rete CAN (`can_send(CAN_VEHICLE, &adc_frame)`) poi serializzato e trasmesso in streaming MQTTS verso HiveMQ, insieme ad altri dati nello stesso pacchetto senza determinare alcun impatto sulle prestazioni computazionali dello stack di rete su Core 0.

3.3.6 Pipeline di Trasmissione: Router e Broker HiveMQ

La trasmissione wireless adotta la seguente configurazione: l'ESP32-S3 si connette via Wi-Fi (protocollo 802.11 b/g/n a 2.4 GHz) a un router automotive 4G-LTE installato a bordo, associandosi come client Station (STA). I pacchetti telemetrici vengono formattati in strutture JSON compatte sfruttando il componente nativo `esp-json` e trasmessi al broker cloud enterprise "HiveMQ" mediante il protocollo "MQTTS" (MQTT cifrato su layer TLS/SSL, porta standard 8883).

```

1 void process_telemetry_frame(void) {
2
3     cJSON *root = cJSON_CreateObject(); // Strutturazione del payload tramite
        componente nativo esp-json
4     cJSON_AddNumberToObject(root, "pack_voltage", car_data.can_v.pack_voltage);
5     cJSON_AddNumberToObject(root, "soc", car_data.can_v.soc);
6     cJSON_AddNumberToObject(root, "pack_current", car_data.can_v.pack_current);
7     cJSON_AddNumberToObject(root, "apps2", car_data.can_v.apps2);
8     cJSON_AddNumberToObject(root, "apps1", car_data.can_v.apps1);
9     cJSON_AddNumberToObject(root, "vx", car_data.can_v.vx);
10
11     char *data_json = cJSON_PrintUnformatted(root);
12     // Pubblicazione asincrona non bloccante verso il broker MQTTS
13     esp_mqtt_client_publish(mqtt_client, "/telemetry", data_json, 0, 0, 0);
14 }

```

Figura 3.6. Esempio di serializzazione e pubblicazione asincrona del payload telemetrico con QoS 0.

La rappresentazione testuale consente l'immediata interoperabilità con l'infrastruttura di terra, aggregando in un unico pacchetto metriche sincronizzate temporalmente. I messaggi di marcia adottano il livello di servizio "QoS 0" (*at most once*), evitando code di ritrasmissione e rispettando la regola FSAE D 2.2.1 sulla telemetria unidirezionale.

Vantaggi dell'architettura:

- MQTT mantiene una singola connessione TCP aperta con un header compatto (appena 2 byte per messaggio base);
- Il sistema ristabilisce da solo le sottoscrizioni e i flussi di pubblicazione senza riavviare la MCU;
- Unico topic con payload JSON aggregato, evitando la frammentazione dei dati su più topic e riducendo il numero di pacchetti;
- La gestione dei certificati TLS è automatizzata e gestita dal framework ESP-IDF;
- La scelta di HiveMQ motivata dalla possibilità di scalare il numero di client sottoscritti e il suo limite di dati mensili ampio.

3.3.7 Infrastruttura di Terra e Visualizzazione su Foxglove Studio

Ai box, un middleware asincrono in Python basata su `asyncio` e librerie `paho-mqtt` intercetta i topic da HiveMQ e li inoltra a un server WebSocket conforme allo standard di Foxglove Studio. Gli ingegneri di pista possono così monitorare contemporaneamente, da più postazioni, i dati telemetrici della vettura, visualizzando grafici in tempo reale e salvando le sessioni direttamente in formato standard MCAP.

Il middleware agisce da traduttore asincrono trasparente: intercetta i messaggi pubblicati dalla vettura su HiveMQ, associa a ciascun topic un canale logico Foxglove conforme allo schema JSON, e trasmette i dati serializzati via WebSocket a bassissima latenza verso tutte le sessioni attive al muretto box.

```
1 import asyncio
2 import time
3 import paho.mqtt.client as mqtt
4 from foxglove_websocket.server import FoxgloveServer
5
6 def on_message(client, userdata, msg):
7     topic = msg.topic
8     asyncio.run_coroutine_threadsafe(server_instance.send_message(channel_map[
9         topic], time.time_ns(), msg.payload))
10
11 def on_connect(client, userdata, flags, reason_code, properties):
12     if reason_code == 0:
13         client.subscribe(MQTT_SUBSCRIBE_TOPIC)
14
15 async def main():
16     async with FoxgloveServer("localhost", 8765) as server:
17         server_instance = server
18         mqtt_client = mqtt.Client()
19         mqtt_client.username_pw_set(MQTT_ID, MQTT_PW)
20
21         mqtt_client.on_message = on_message
22         mqtt_client.on_connect = on_connect
23         mqtt_client.connect(MQTT_BROKER, MQTT_PORT, 60)
24         mqtt_client.loop_start()
25
26         while True:
27             await asyncio.sleep(1)
28
29 if __name__ == "__main__":
30     asyncio.run(main())
```

Figura 3.7. Esempio di implementazione del FOXGLOVE Bridge (MQTTTS - WebSocket)

Grazie al disaccoppiamento ottenuto tramite il broker HiveMQ e la serializzazione asincrona su Python, l'architettura garantisce la massimizzazione del throughput informativo: la vettura immette i vettori di campionamento sul canale di trasmissione a cadenza costante di "20 ms" (frequenza nominale di 50 Hz), permettendo a più ingegneri di pista di esaminare simultaneamente i parametri veicolo in tempo reale da postazioni indipendenti senza generare alcun carico computazionale o interferenza sul nodo embedded di bordo.

3.4 Interfaccia Pilota (HMI): Volante e Dashboard

L'HMI consente al pilota di selezionare mappe motore e setup di guida, fornendo al contempo una visione immediata delle grandezze critiche senza distrazioni. Nel lavoro svolto, l'architettura HMI è stata integralmente ripensata attraverso la riprogettazione della scheda elettronica del volante (*Steering Wheel PCB*), lo sviluppo della nuova scheda di controllo della Dashboard (basata su microcontrollore ATmega328P) e l'ottimizzazione del layout grafico sul display intelligente Nextion da 5 pollici.

3.4.1 Scheda Dashboard su ATmega328P

La scheda Dashboard si occupa della gestione dei LED di allarme, della lettura dei sensori anteriori e della trasmissione dei dati sul display Nextion. Il sistema:

- ospita un microcontrollore " ATmega328P" collegato alla dorsale CAN tramite controller MCP2515, con comunicazione via SPI e ricetrasmittitore TJA1050;
- usa uno stadio di alimentazione uguale a quella della Auxiliary Control Board, con protezione da inversione di polarità e soppressione dei transienti condotti e integra filtri per entrambi gli ingressi digitali e analogici;
- pilota le tre spie di sicurezza regolamentari (BMS, IMD, BSPD) tramite MOSFET a canale N a basso assorbimento montati su uscite dedicate e su scheda per testing;

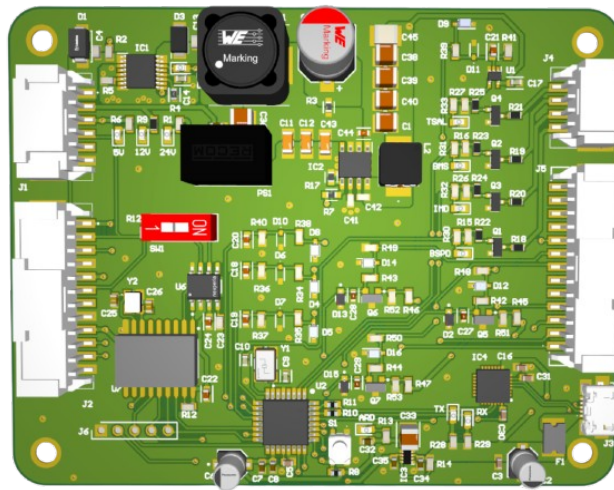


Figura 3.8. Render 3D della scheda Dashboard

- configura filtri e maschere hardware nel controller ($\text{MASK_CAN1} = 0x500$, $\text{MASK_CAN2} = 0x700$) per rigettare il traffico di fondo e processare unicamente i frame di interesse;
- acquisisce tramite ADC a 10 bit l'angolo di sterzo e le corse delle sospensioni anteriori, inviandole sulla rete veicolo nel frame CAN 0x420;

- gestisce gli allarmi tramite interrupt sui pin (PCINT2) con manipolazione diretta dei registri PIND e PORTD, garantendo la commutazione immediata delle spie in tempo reale.

```

1 // Aggiornamento atomico delle spie di allarme su registro PORTD (ISR)
2 inline void aggiornaUscitePortD() {
3     byte statoPIND = PIND; // Lettura diretta dello stato degli ingressi
4     // Pin 6 (PD6, IMD) LOW -> Uscita Pin 3 (PD3, IMD_LED) HIGH (e viceversa)
5     if (!(statoPIND & (1 << PD6))) {
6         PORTD |= (1 << PD3);
7     } else {
8         PORTD &= ~(1 << PD3);
9     }
10    // Pin 7 (PD7, BMS) LOW -> Uscita Pin 4 (PD4, BMS_LED) HIGH (e viceversa)
11    if (!(statoPIND & (1 << PD7))) {
12        PORTD |= (1 << PD4);
13    } else {
14        PORTD &= ~(1 << PD4);
15    }
16 }
17 ISR(PCINT2_vect) {
18     aggiornaUscitePortD();
19 }

```

Figura 3.9. Routine di gestione diretta dei registri per lo switch delle spie di sicurezza all'interno della ISR.

Come illustrato nel Codice 3.9, l'elaborazione avviene a basso livello mediante accesso diretto ai registri PIND e PORTD senza ricorrere a funzioni software di libreria bloccanti. Questo garantisce che la disattivazione di una linea di allarme (logica attiva-bassa) commuti istantaneamente la spia corrispondente con latenza sub-microsecondo.

Procedura di Bulb-Check all'Avvio (T 11.9.6): In conformità con il regolamento, la funzione `ledSetup()` forza all'avvio del LVS una sequenza temporizzata di test ottico: tutti gli indicatori SCS vengono mantenuti accesi per 1500 ms, disattivati per 1000 ms e riaccesi per ulteriori 500 ms prima di abilitare le routine di interrupt ordinarie. Ciò consente ai commissari tecnici di verificare l'efficienza dei cablaggi e delle connessioni dei LED di sicurezza prima della partenza della monoposto.

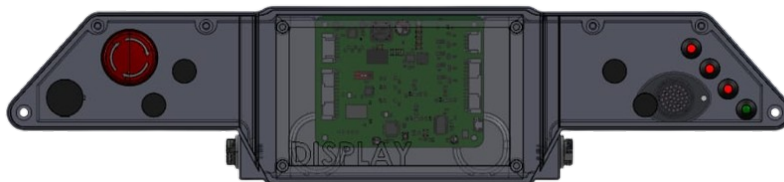


Figura 3.10. Render 3D del cockpit con Scheda Dashboard all'interno.

3.4.2 Scheda Volante con Modulo CFE18

La versione precedente del volante integrava un microcontrollore e un modulo CAN Plug and Play ed ogni pin del pulsante o selettore era collegato a un pin del microcontrollore. La nuova scheda adotta il modulo compatto "CFE18 CAN Switch-board", che digitalizza i comandi su CAN bus a 500 kbps, mantenendo comunque i pulsanti e i selettori rotativi già presenti nella versione precedente.

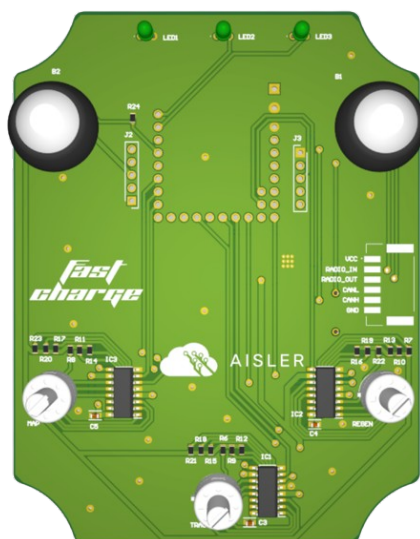


Figura 3.11. Assemblaggio del volante: pulsantiere, selettori rotativi e scheda con modulo CFE18.

I selettori rotativi multigiro (dedicati a freno rigenerativo, mappe motore e traction control) sono interfacciati a dei SN74HC148DR priority encoder. Questo per poter mappare 6 posizioni logiche su 3 linee digitali, per 3 selettori distinti.

Tabella 3.3. Decodifica dei selettori rotativi tramite il modulo CFE18 attraverso il priority encoder SN74HC148DR (logica invertita).

Pos	In 0	In 1	In 2	In 3	In 4	In 5	bit2	bit1	bit0
1	0	1	1	1	1	1	1	1	0
2	1	0	1	1	1	1	1	0	1
3	1	1	0	1	1	1	1	0	0
4	1	1	1	0	1	1	0	1	1
5	1	1	1	1	0	1	0	1	0
6	1	1	1	1	1	0	0	0	1

Il convertitore CFE18 campiona le linee risultanti e le trasmette sul Vehicle CAN nei messaggi dedicati SW1 (0x700), SW2 (0x701) e SW3 (0x702). Il compito poi di decodificare i bit di posizione dei selettori rotativi è affidato alla scheda Dashboard.

```

1 // Decodifica dello stato logico dei selettori rotativi (Dashboard ATmega328P)
2 case ST_WHEEL_ID_1: // CAN ID 0x700 (SW1)
3     v = ((uint16_t)(canMsgRead.data[0] << 8) & 0xFF00) | canMsgRead.data[1];
4     bit0_s1 = (v > 512) ? 0 : 1; // il valore di 0 logico corrisponde a 1023,
        mentre il valore di 1 logico corrisponde a 0
5     v = ((uint16_t)(canMsgRead.data[2] << 8) & 0xFF00) | canMsgRead.data[3];
6     bit1_s1 = (v > 512) ? 0 : 1;
7     v = ((uint16_t)(canMsgRead.data[4] << 8) & 0xFF00) | canMsgRead.data[5];
8     bit2_s1 = (v > 512) ? 0 : 1;
9     v = ((uint16_t)(canMsgRead.data[6] << 8) & 0xFF00) | canMsgRead.data[7];
10    bit0_s2 = (v > 512) ? 0 : 1;
11
12    setting = (bit0_s1) + (2 * bit1_s1) + (bit2_s1 * 4);
13    break;

```

Figura 3.12. Estrazione e ricostruzione dei bit di posizione dei selettori rotativi dai frame generati dal CFE18.

3.4.3 Interfaccia Grafica su Display Nextion 5"

La visualizzazione a display adotta un pannello HMI a colori Nextion Intelligent Series da 5". La comunicazione tra l'ATmega328P e il MCU grafico del Nextion avviene su linea seriale "UART" a 115200 BAUD, inviando comandi formattati in stringhe ASCII terminate dal terminatore convenzionale a tre byte (`\xFF\xFF\xFF`).

- **Pagina di Gara principale:** visualizza la velocità istantanea, il SoC percentuale con indicatore progressivo, la tensione di pack e le temperature aggregate di motori, inverter e batteria (con commutazione da verde a rosso sopra i 60 °C);
- **Overlay Temporaneo di Setup:** quando il pilota ruota un selettore, appare una schermata con il nuovo valore selezionato per 850 ms, dopo di che lo schermo torna automaticamente alla pagina di gara;
- **Gestione Missioni Driverless:** a monoposto ferma, consente di impostare la missione autonoma desiderata (Skidpad, Trackdrive, Acceleration) e comandare l'AS Computer via CAN.

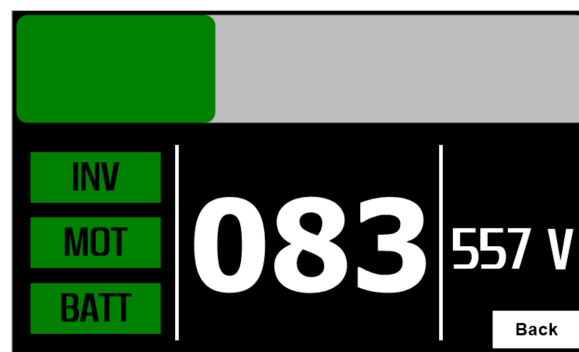


Figura 3.13. Interfaccia di gara sul display Nextion

Grazie all'introduzione della nuova telemetria live, tutti i dati non operativi sono stati eliminati dal cruscotto. Il pilota riceve esclusivamente le informazioni critiche per la guida e la gestione strategica dell'energia.

3.5 Sistema Sperimentale di Acquisizione Aero Rake

L'apparato *Aero Rake* è stato progettato per mappare sperimentalmente il profilo delle pressioni e la scia turbolenta generata dallo pneumatico anteriore, permettendo la validazione dei modelli CFD elaborati dal reparto aerodinamica, più precisamente dalla tesi del mio collega [10].

A differenza dei nodi primari di bordo (AMS e VCU), l'Aero Rake non svolge funzioni di controllo attivo né presenta requisiti stringenti di sicurezza funzionale. Pertanto, il sistema è stato concepito secondo un approccio modulare e compatto basato sul framework Arduino, eliminando il bisogno di uno sviluppo di un'architettura bare-metal complessa a fronte di un carico computazionale ridotto.

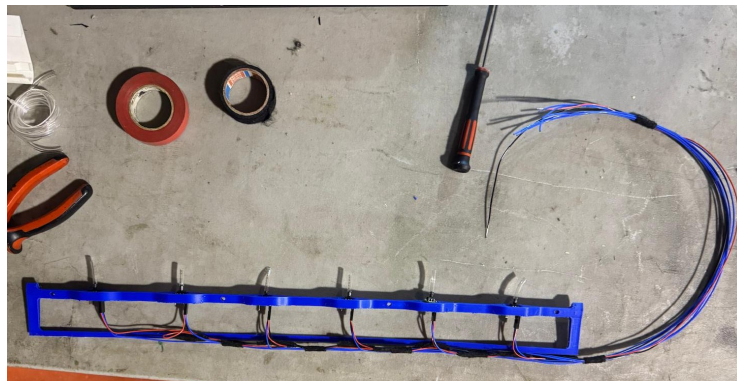


Figura 3.14. Elemento/Rastrello singolo dell'Aero Rake, con 6 sensori di pressione differenziale e cablaggio integrato.

3.5.1 Sensoristica e Hardware di Acquisizione

L'elettronica di un singolo segmento dell'Aero Rake (l'intero sistema ne contiene 5) include:

- 6 sensori di pressione piezoresistivi differenziali commerciali (CFSensor XGZP6847), con fondo scala di ± 1 kPa e uscita ratiometrica centrata a 2.5 V per $\Delta P = 0$ Pa;
- un microcontrollore "ATmega32U4" (Arduino Micro) che campiona direttamente le uscite ratiometriche tramite il proprio convertitore ADC a 10 bit alimentato a 5 V;
- un modulo Plug-And-Play MCP2515 con oscillatore a 8 MHz, collegato via bus SPI, che trasmette le misure a 500 kbps.

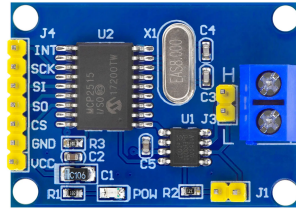


Figura 3.15. Modulo Plug-and-Play Commerciale MCP2515.

3.5.2 Firmware e Integrazione CAN

L'architettura software è sviluppata in ambiente Arduino. Non richiedendo macchine a stati complesse né la gestione di comunicazioni bidirezionali asincrone, il ciclo di esecuzione adotta un superloop temporizzato non bloccante governato dalla libreria `mcp_can.h`:

```

1  MCP_CAN CAN0(PIN_CS); // Istanza del driver MCP2515 con pin CS dedicato
2
3  void setup() {
4      CAN0.begin(CAN_500KBPS);
5  }
6
7  void loop() {
8      unsigned int val1 = map(analogRead(PIN_SIG1), 0, 1023, 0, 4999);
9      // ... analogRead per tutti i 6 sensori
10
11     // Impacchettamento in formato Big-Endian (MSB first)
12     unsigned char buf1[6] = {
13         (unsigned char)((val1 >> 8) & 0xFF), (unsigned char)(val1 & 0xFF),
14         (unsigned char)((val2 >> 8) & 0xFF), (unsigned char)(val2 & 0xFF),
15         (unsigned char)((val3 >> 8) & 0xFF), (unsigned char)(val3 & 0xFF)
16     };
17     unsigned char buf2[6] = {
18         (unsigned char)((val4 >> 8) & 0xFF), (unsigned char)(val4 & 0xFF),
19         (unsigned char)((val5 >> 8) & 0xFF), (unsigned char)(val5 & 0xFF),
20         (unsigned char)((val6 >> 8) & 0xFF), (unsigned char)(val6 & 0xFF)
21     };
22
23     CAN0.sendMsgBuf(ID_SEGMENT1, 0, 6, buf1);
24     CAN0.sendMsgBuf(ID_SEGMENT2, 0, 6, buf2);
25     delay(DELAY_MS); // Delay per rispettare la frequenza di campionamento
                       // desiderata
26 }

```

Figura 3.16. Logica di implementazione del firmware di un segmento dell'Aero Rake.

Grazie all'integrazione sul Vehicle CAN, le pressioni acquisite risultano intrinsecamente sincronizzate con la velocità ruota e le accelerazioni registrate dalla VCU, permettendo un'immediata correlazione con le simulazioni numeriche.

Capitolo 4

Validazione Sperimentale e Risultati

Tutti i sottosistemi realizzati sono stati collaudati secondo un percorso a tre fasi:

1. **Verifica funzionale a banco:** Controllo dei tempi di reazione dei sistemi, test di iniezione di guasto (*fault injection*) e verifica dei requisiti regolamentari di tempo;
2. **Ispezioni tecniche ufficiali (Formula Student Inspection):** Validazione sulle specifiche di regolamento dell'accumulatore e dei sistemi di sicurezza durante gli eventi ufficiali di *Formula Student Spain* e *Formula SAE Italy*;
3. **Validazione dinamica in pista e telemetria live:** Acquisizione continua dei dati durante le prove dinamiche sul circuito di Varano de' Melegari, con archiviazione dei log in formato standard MCAP e confronto fluidodinamico del modulo Aero Rake.



Figura 4.1. La monoposto durante le ispezioni, Formula SAE Italy 2026.

4.1 Validazione a Banco e Testing dell'AMS

L'Accumulator Management System è stato sottoposto a una severa procedura di calibrazione a banco prima dell'integrazione definitiva all'interno del pacco batterie.

- Il funzionamento dell'AMS è stato verificato usando dei segmenti "TESTER" dell'accumulatore, con connettori di test per determinare il funzionamento della rilevazione dei guasti e dei tempi di intervento.
- sono stati controllati il funzionamento della gestione degli stati dell'AMS, la validità dei dati letti e logica complessiva, attraverso il feedback su CAN, e lo sgancio dei contattori AIR (OPEN_ALL_AIR()) [6].

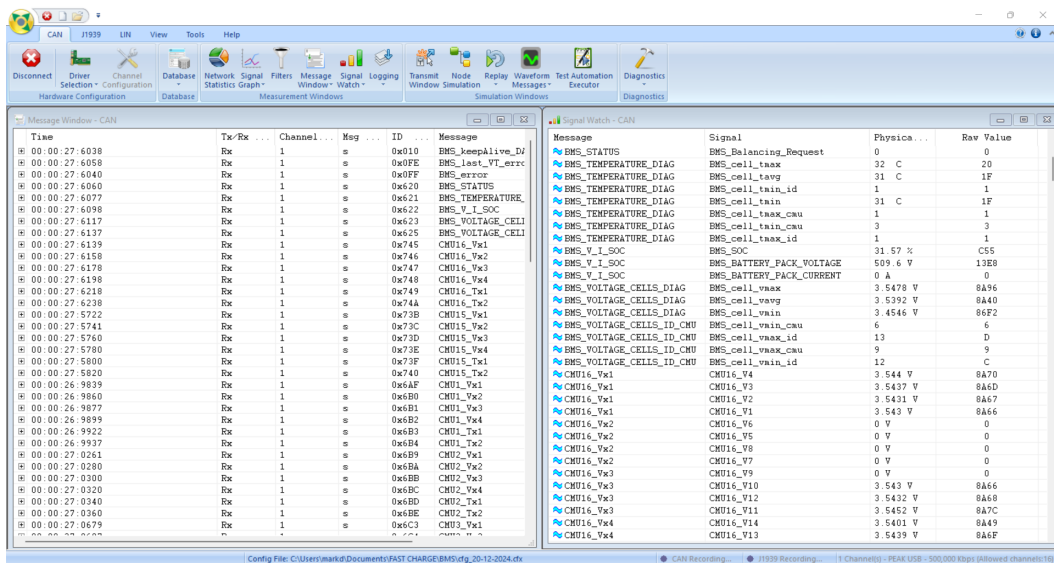


Figura 4.2. Diagnostica CAN su Software BUSMASTER, con tutti i dati dell'AMS, come richiesto anche dal Regolamento

- I tempi registrati rispettano pienamente le soglie regolamentari (meno di 500 ms per la media di tensione e meno di 1 s per le temperature).
- Ritardi marginali rispetto alle costanti inserite sono imputabili al tempo di scansione daisy-chain isoSPI dei 16 moduli AFE e alla durata del superloop generale.

Una volta integrato l'SPC58EC all'interno del contenitore dell'accumulatore reale (16 CMU collegate in daisy-chain isoSPI), i test sono stati replicati a vettura ferma, confermando l'assoluta stabilità delle letture.



Figura 4.3. Il pacco batterie con l'AMS integrato e le 16 schede AFE collegate in daisy-chain isoSPI, accesi e in comunicazione

4.1.1 FSAE Inspections

Nella competizione "Formula SAE Italy", il pacco batterie ha superato con esito positivo le verifiche dei commissari tecnici:

- durante la prova di *Open-Wire Detection* (scollegamento forzato di un conduttore di sensing di tensione), il firmware ha rilevato l'anomalia aprendo l'SDC e bloccando il sistema nello stato **ERROR** con latching permanente fino al riavvio manuale;
- la monoposto ha completato con successo le ispezioni tecniche (meccaniche e elettriche), pioggia (*Rain Test*) e frenata d'emergenza (*Brake Test*).



Figura 4.4. La monoposto sul tracciato di Varano de' Melegari durante le prove dinamiche.

La monoposto ha superato con esito pienamente positivo l'*Electrical Inspection*, l'*Accumulator Inspection* e i successivi test funzionali di *Rain Test* e *Brake Test*, ottenendo il pass ufficiale per l'accesso a tutti gli eventi dinamici della competizione.

4.2 Prestazioni della Telemetria in Pista

Durante le prove dinamiche sul circuito di Varano de' Melegari, il sistema di telemetria ha dimostrato un funzionamento stabile e affidabile, sia per l'analisi dei dati in tempo reale sia per rendere partecipi i membri del team non presenti in pista.

I principali risultati ottenuti sono:

- Il SoC ESP32-S3 ha mantenuto un'emissione ininterrotta dei pacchetti MQTTS a circa 20 ms. Inoltre i flussi pubblicati dalla monoposto e letti dal Bridge Python sono stati registrati direttamente in MCAP, oppure attraverso la funzione di Foxglove Studio, in CSV, per i singoli dati/plot.



Figura 4.5. Visuale della scheda ACB montata nella monoposto, con il modulo ESP32-S3.

- La visualizzazione in tempo reale su Foxglove Studio ha consentito di monitorare simultaneamente più parametri della monoposto, con la possibilità di salvare le sessioni in formato standard MCAP per successive analisi. I dati acquisiti hanno mostrato un andamento coerente con le aspettative e con l'andamento della monoposto stessa.



Figura 4.6. Sessione di telemetria live acquisita durante una run a Varano: andamento coordinato di corrente erogata, tensione di pack, velocità e SoC, con visualizzazione in tempo reale su Foxglove Studio.

4.3 Risultati dell'Interfaccia Cockpit e Volante

I test in vettura hanno validato il funzionamento dell'HMI:



Figura 4.7. Visuale dell'abitacolo con la nuova Dashboard e il volante installati a banco.

- la sequenza di bulb-check all'accensione è stata convalidata;
- la codifica dei selettori sul volante con modulo CFE18 ha consentito una selezione precisa, risultato visibile anche attraverso il setting mostrato a schermo;
- l'eliminazione dei dati superflui dal display Nextion a favore di indicatori sintetici ha ridotto il carico di attenzione del pilota; la visualizzazione temporanea di 850 ms per il cambio mappa si è rivelata utile.



Figura 4.8. Visuale dell'abitacolo con la nuova Dashboard e il volante installati sulla vettura.

4.4 Risultati del Sottosistema Aero Rake

Il sistema sperimentale Aero Rake è stato collaudato a velocità costanti di 10 m/s e 15 m/s su tratti rettilinei.



Figura 4.9. Foto della Aero Rake montata sul veicolo [10].

Le tensioni acquisite dai sensori di pressione a 100 Hz sono state convertite nel coefficiente di pressione totale adimensionale:

$$C_{pt} = \frac{P_{tot} - P_{\infty}}{\frac{1}{2}\rho v_{\infty}^2} \quad (4.1)$$



Figura 4.10. Dettaglio della Aero Rake montata sul veicolo.

I risultati sperimentali hanno mostrato un buon accordo con le simulazioni numeriche fluidodinamiche CFD (STAR-CCM+) elaborate nella tesi di laurea magistrale dell'Ing. De Marco [10], individuando con precisione le scie e il vortice generato dal rotolamento della ruota anteriore.

Capitolo 5

Conclusioni e Sviluppi Futuri

Il lavoro descritto in questa tesi ha consentito di riprogettare, implementare e validare sperimentalmente i principali sottosistemi di diagnostica, acquisizione e telemetria per la monoposto elettrica della scuderia universitaria.

5.1 Linee Guida per gli Sviluppi Futuri

La tesi è incentrata su progetti appartenenti ad un'un'architettura modulare e scalabile, i quali sia presi singolarmente sia integrati tra loro, offrono un ampio margine di sviluppo per una ulteriore ottimizzazione, rigidità e precisione delle funzionalità implementate. Tra le possibili linee guida per i futuri sviluppi si segnalano:

- **Stima del SoC mediante Filtro di Kalman Esteso (EKF):** l'attuale approccio combinato OCV-Coulomb Counting può essere esteso integrando un modello elettrico equivalente di cella (*Thevenin model*) su osservatore EKF, per stimare dinamicamente anche la resistenza interna.
- **Mantenimento del Bare-Metal su SPC58:** Date le complessità dell'ambiente SPC5Studio e le risorse computazionali vincolate, l'architettura bare-metal a superloop deterministico rimane la scelta ottimale. Essa garantisce il controllo assoluto dei tempi di esecuzione.
- **Motore di Serializzazione Dinamica:** L'attuale codifica JSON statica richiede modifiche firmware per ogni variazione dei segnali CAN. È opportuno introdurre una struttura a descrittori o tabelle di parsing dinamiche caricate in memoria, consentendo l'aggiunta di nuovi ID diagnostici senza ricompilare l'intero applicativo.
- **Architettura Dual-MCU per la Telemetria:** separare fisicamente su due microcontrollori le funzioni di I/O a bassa latenza e la gestione dello stack radio Wi-Fi/cellulare (ad esempio tramite driver `esp-hosted` su bus SPI/SDIO);
- **Aggiornamenti Firmware Over-The-Air (OTA):** abilitare il partizionamento Flash Dual-OTA sull'ESP32-S3 per consentire la riprogrammazione remota del gateway telemetrico via 4G direttamente dai box durante i test;

5.2 Considerazioni Personali

L'esperienza all'interno della scuderia Sapienza Fast Charge ha rappresentato un percorso formativo importante, offrendo l'opportunità di applicare i principi teorici imparati e poterli applicare su un sistema ad alta complessità, imparando e padroneggiare competenze non puramente legate al mio corso di studi. L'esperienza ha permesso di acquisire competenze trasversali, anche oltre a quelle strettamente tecniche, come la gestione del lavoro in team, la pianificazione delle attività e la risoluzione dei problemi.

Il lavoro sul campo — dalla programmazione a basso livello alla progettazione di PCB, fino al collaudo nei box di gara — mi ha permesso di acquisire un metodo di lavoro strutturato, orientato all'affidabilità e al soddisfacimento di, anche severi, vincoli di sicurezza. L'architettura documentata in questo lavoro lascia al team un'infrastruttura solida, modulare e pronta per i successivi sviluppi.



Figura 5.1. Sapienza Fast Charge, Formula Student Spain 2026.

Bibliografia

- [1] Formula Student Germany, *Formula Student Rules 2026*, Version 1.1, 2026.
https://www.formulastudent.de/fileadmin/user_upload/all/2026/rules/FS-Rules_2026_v1.1.pdf
- [2] N. Navet, Y. Song, F. Simonot-Lion, and C. Wilwert, “Trends in automotive communication systems,” *Proceedings of the IEEE*, vol. 93, no. 6, pp. 1204–1223, 2005.
<https://doi.org/10.1109/JPROC.2005.849725>
- [3] M. Brandl et al., “Batteries and battery management systems for electric vehicles,” in *2012 IEEE International Electric Vehicle Conference (IEVC)*, Greenville, SC, USA, 2012, pp. 1–6.
<https://doi.org/10.1109/IEVC.2012.6183296>
- [4] G. L. Plett, “Extended Kalman filtering for battery management systems of LiPB-based HEV battery packs: Part 1. Background,” *Journal of Power Sources*, vol. 134, no. 2, pp. 252–261, 2004.
<https://doi.org/10.1016/j.jpowsour.2004.02.031>
- [5] N. K. Gondhi and C. Sharma, “Communication Protocol Stack for Constrained IoT Systems,” in *2018 3rd International Conference On Internet of Things: Smart Innovation and Usages (IoT-SIU)*, Bhimtal, India, 2018, pp. 1–5.
<https://doi.org/10.1109/IoT-SIU.2018.8519904>
- [6] STMicroelectronics, *SPC58 C Line - 32-bit Power Architecture Automotive Microcontroller Family Reference Manual*, DocID RM0407, Rev. 4, 2023.
- [7] STMicroelectronics, *L9963E: Automotive Chip for Battery Management and Cell Monitoring*, Datasheet DS13352, Rev. 3, 2021.
- [8] Espressif Systems, *ESP32-S3 Technical Reference Manual*, Version 1.4, 2023.
- [9] Microchip Technology, *MCP2515: Stand-Alone CAN Controller with SPI Interface*, Datasheet DS21801H, 2019.
- [10] M. De Marco, “Analisi aerodinamica di una vettura Formula SAE mediante CFD e validazione sperimentale,” Tesi di Laurea Magistrale in Ingegneria Meccanica, Facoltà di Ingegneria Civile e Industriale, Sapienza Università di Roma, A.A. 2024/2025.

-
- [11] Lithium Balance, *N-BMS: Battery Management System for Electric Vehicles*, BMS Manual Version 2.4, 2018.